

Ordenação de Grandes Volumes de Dados Genômicos usando MPI

Trabalho 1 - Programação Paralela

Maryana Catonho

- **Introdução**

O relatório apresenta e demonstra a implementação de um sistema de ordenação paralela de grandes volumes de DNAs (sequências genômicas) utilizando MPI (Message Passing Interface) para comparar o desempenho entre paralela e sequencial.

- **Estratégia adotada**

A estratégia escolhida para a ordenação paralela foi utilizar o algoritmo Split Sort em C com MPI; a distribuição dos dados possui o rank 0 para leitura de arquivos de entrada com o total de sequências N sendo transmitido para todos os processos, sendo todas distribuídas igualmente entre os P processos com MPI_Scatterv. Cada processo recebe assim um conjunto de dados e ordena com qsort. Os dados são redistribuídos com o MPI_Alltoall, então, cada processo ordena seu novo bucket localmente. Os dados são coletados com MPI_Gatherv para escrever em um novo arquivo de saída com eles organizados.

Foram usados para teste arquivos de 100K, 1M e 10M sequências com 2, 4 e 8 processadores. Cada arquivo de entrada foi gerado com o arquivo “gerar.c” (mais informações no README)

• Resultados

Os resultados foram muito satisfatórios

GRÁFICO 1 - comparação sequencial X paralela com 2 processadores

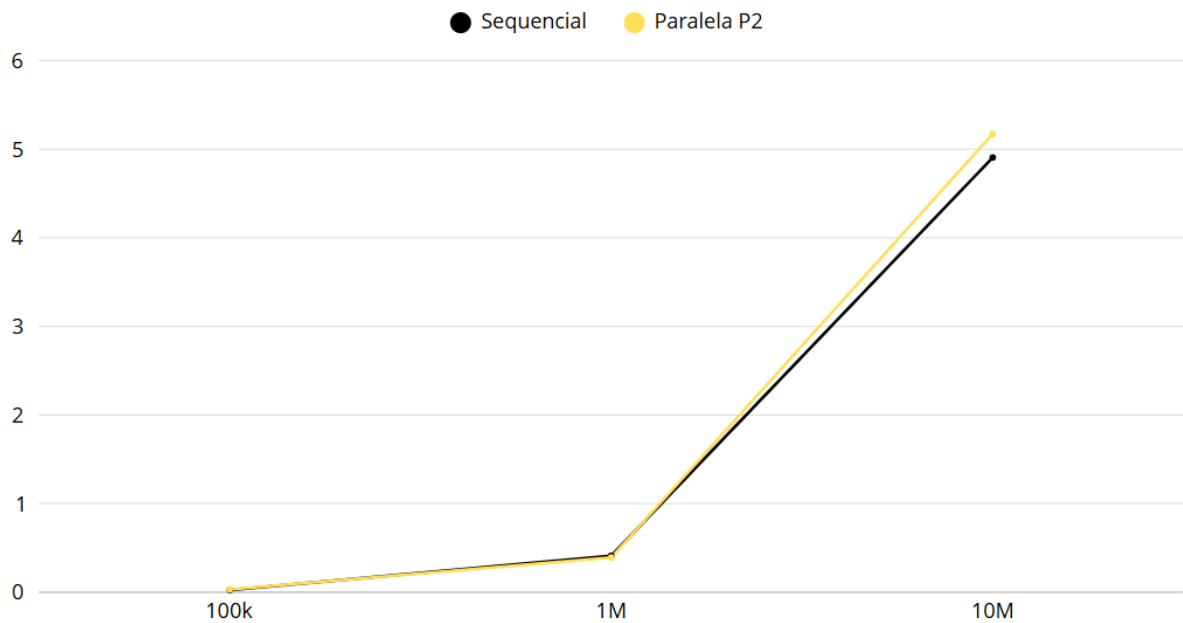


GRÁFICO 2 - comparação sequencial X paralela com 4 processadores

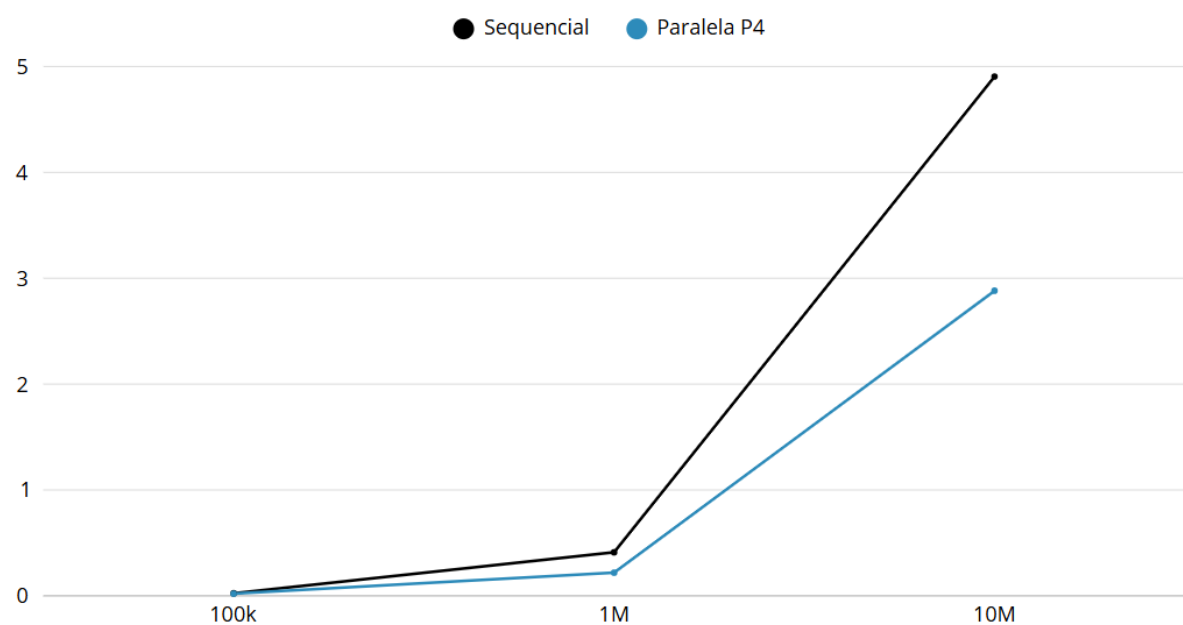


GRÁFICO 3 - comparação sequencial X paralela com 8 processadores

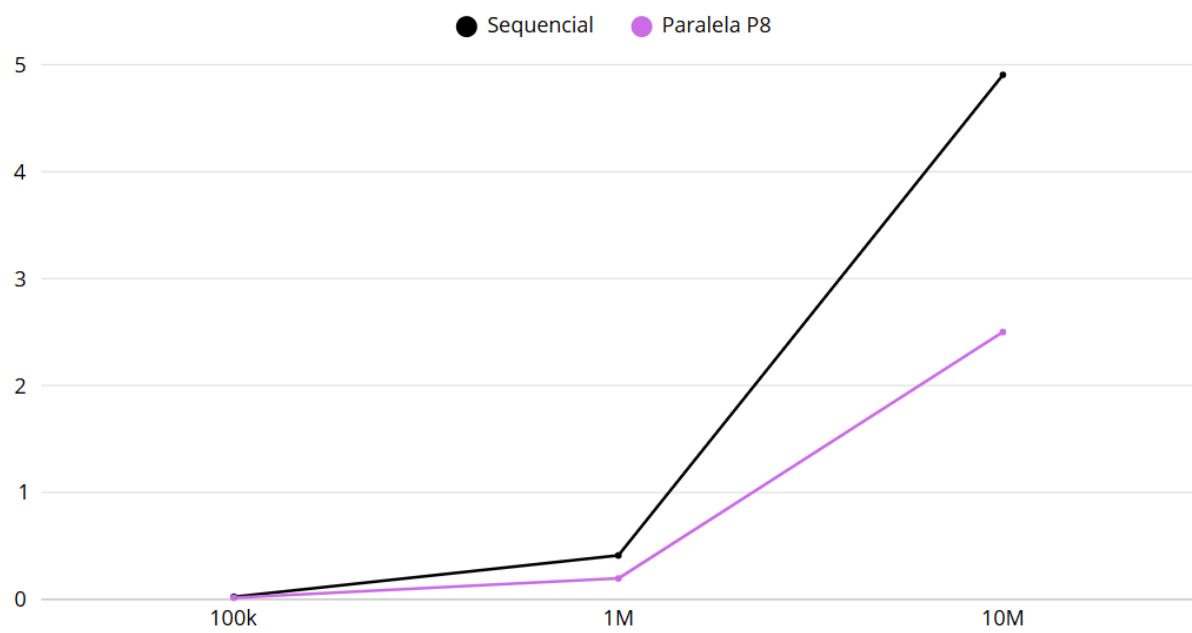


GRÁFICO 4 - comparação de todos

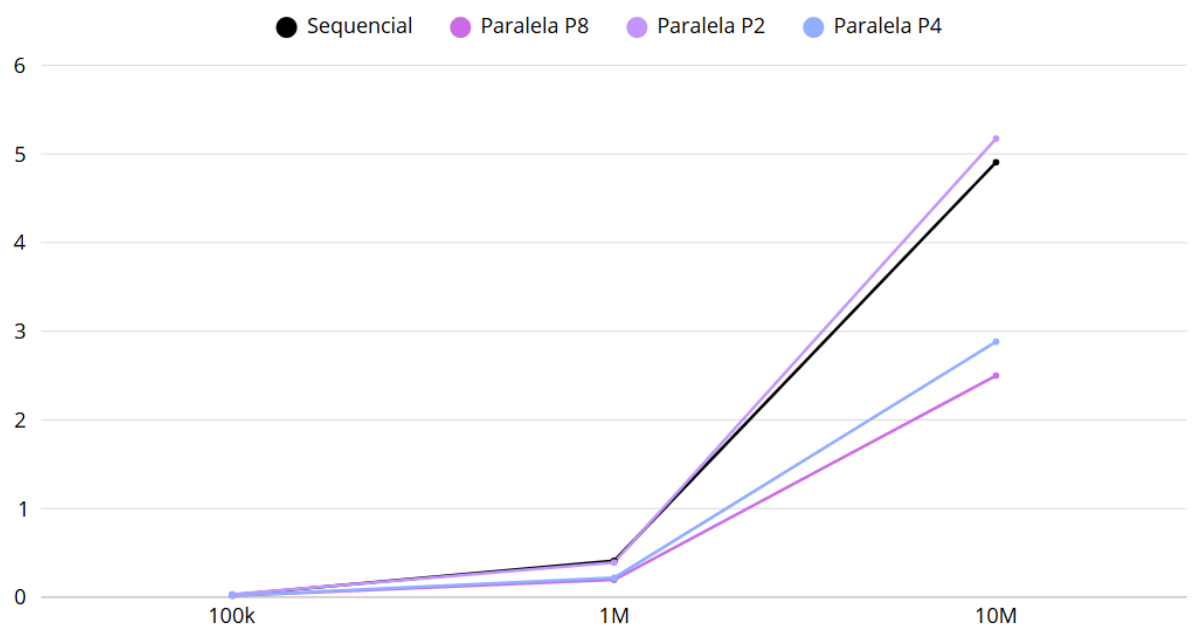


GRÁFICO 5 - Speedup ($T_{seq}(n)/T_{par}(n)$)

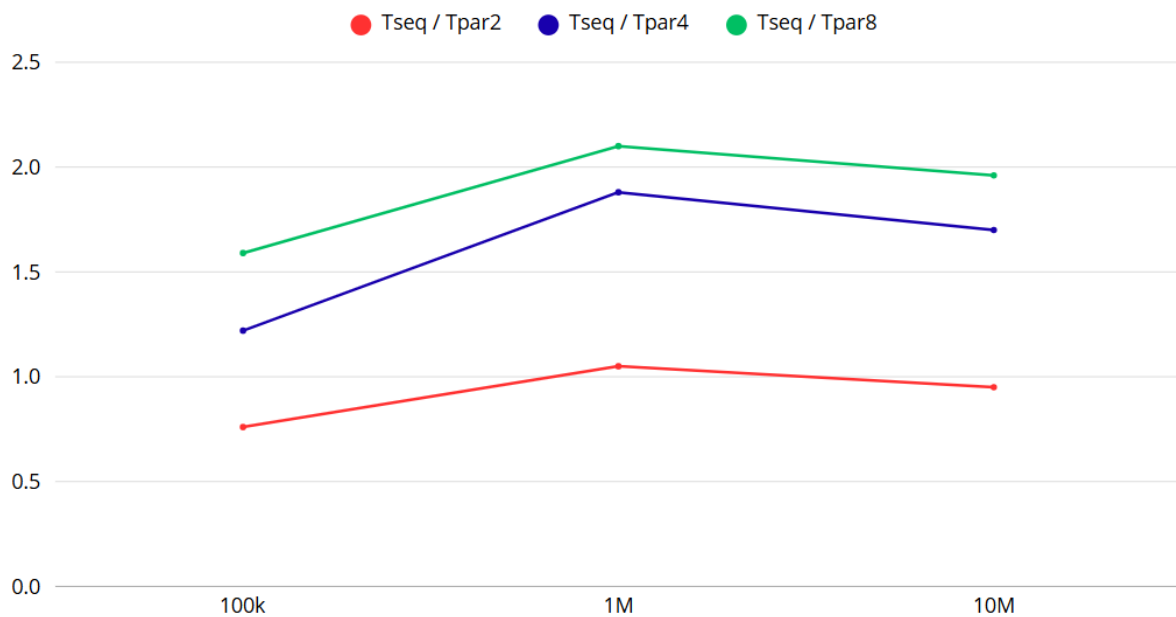


TABELA 1 - demonstração de todos os números

	Sequencial	Paralela P2	Paralela P4	Paralela P8
100K	0.024442s	0.032171s	0.019959s	0.015401s
1M	0.41091s	0.391399s	0.218472s	0.195751s
10M	4.906266s	5.172903s	2.882413s	2.499986s

TABELA 2 – Speedup ($T_{seq}(n)/T_{par}(n)$)

	Tseq / Tpar2	Tseq / Tpar4	Tseq / Tpar8
100K	0.76s	1.22s	1.59s
1M	1.05s	1.88s	2.10s
10M	0.95s	1.70s	1.96s

• Análise

Como esperado, os resultados com paralelismo foram (quase) todos melhores que o sequencial.

Com 2 processadores em 100k sequências o tempo do paralelo foi maior que o sequencial, com o speedup menor que 1, ou seja, não houve melhora de desempenho, mas em 4 e 8 processadores, o speedup melhora, demonstrando o ganho de aproveitamento.

Em 1M e 10M é nítido a melhoria; em 10M e 8 processadores o tempo saiu de quase 5 segundos pra 2.5 segundos com speedup de quase 2 e em 1M foi ainda melhor com speedup de 2.10!

- **Ganho de desempenho**

Com o cálculo do speedup é possível comprovar o ganho do desempenho, tendo visão melhor a partir do uso de 2 processadores em 1M de linhas uma vez que possui tamanho grande o suficiente pra usar a comunicação.

Em 100k, o aumento de processadores mostra que, mesmo com entradas pequenas, adicionar mais processadores no processo melhora na rapidez, no caso, com 2 saiu de 0.76 para mais de o dobro com 1.59.

- **Limitações observadas**

A parte onde os dados precisam ser redistribuídos consome maior parte do tempo de processamento (MPI_Alltoall) pois processa a cópia e envio de muitas strings e, dependendo do caso, muito longas também.

Um problema de sincronização também atrapalha no processamento pois os buckets precisam esperar o maior terminar o seu processo para poder continuar.

- **Conclusão**

A implementação da ordenação paralela com MPI é mais eficiente na grande maioria dos casos em comparação da ordenação sequencial para grandes volumes, no caso, de dados genômicos, tendo o uso de 8 processadores mais vantajoso, com melhoria sendo 2 vezes mais rápido em alguns experimentos.

O ganho do desempenho está ligado ao volume de dados assim como a limitação está relacionada com a grande quantidade de operações que tem que serem feitas, como a redistribuição de dados.